

Ex.No: 01
Date:

Shell Scripting for System Automation

Aim:

To Automate System administration task such as user creation log monitoring and data backup using Bash shell scripting.

Software Requirements:

1. Ubuntu Linux OS
2. Bash Shell
3. Terminal
4. Root / Sudo privileges

Algorithm:

1. Start the process.
2. Create a shell script file using a text editor.
3. Write commands to create a new user.
4. Write commands to perform directory backup.
5. Commit changes to repositories.
6. View repository status and history.
7. Stop the process show the result.

Output:

user Create.sh output

Enter username : Ashok

User already exists

log - monitor.sh output

Feb 20 10:22:31 System[1] : error starting server

Feb 20 10:25:11 Kernel : error detected in module

Feb 20 10:28:44 nginx : error connecting to server

backup.sh output

/tmp/backup-2026-02-23.tar.gz

Program:

user create.sh

#!/bin/bash

echo "Enter username:"

read username

if id "\$username" &>/dev/null

then

echo "User already exists"

else

sudo user add "\$username"

echo "User created successfully"

fi

log monitor.sh

#!/bin/bash

grep -I "error" /var/log/syslog | tail -10

backup.sh

#!/bin/bash

tar -czf /tmp/backup_\$(date +%F).tar.gz /home

Result:

Thus system task such as uses creation log monitoring and data backup using bash shell scripting successfully automated using bash shell scripting.

Ex.No: 02
Date:

Virtual Machine Provisioning using virtual Box and vagrant

Aim:

To set up a virtualise environment using virtualBox automate virtual machine provisioning using vagrant and continue networking and shared folders.

Software Requirements:

1. Ubuntu Linux OS
2. Bash Shell
3. Terminal
4. Root / Sudo privileges

Algorithm:

1. Start the process.
2. Install virtual Box and vagrant.
3. Create a project directory.
4. Initialise vagrant.
5. Configure vagrant file.
6. Start the virtual machine.
7. Configure networking.
8. Configure shared folders.
9. Verify setup.
10. Stop the process show the result.

Output:

Bringing machine 'default' up with 'virtual box' provider....

==> default : Importing base box 'ubuntu/focal64'....

==> default : Setting the name of the VM....

==> default : Configuring private network....

==> default : Mounting shared folders....

==> default : Running provisioner : shell....

 default : updating packages....

 default : Installing nginx....

To verify : <https://192.168.54.40>

Program:

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/focal64"
  config.vm.network "private_network", ip: "192.168.56.40"
  config.vm.synced_folder "./shared", "/vagrant_shared"
  config.vm.provision "shell", inline: <=>SHELL
    sudo apt update
    sudo apt install -y nginx
  SHELL
end
```

Result:

Thus a virtualised environment was successfully created using virtual Box and vagrant with automated networking and shared folder configuration.

Ex.No: 03
Date:

Version Control using Git for Developers

Aim:

To gain hands -on experience with Git by initialising a repository tracking files committing changes and understanding basic version control operations used in.

Software Requirements:

1. Ubuntu Linux / window
2. Git
3. Terminal / command prompt

Algorithm:

1. Start the process.
2. Install and verify Git.
3. Initialise a Git repository.
4. Create and track files.
5. Commit changes to repository.
6. View repository status and history.
7. Stop the process show the result.

Output:

git version 2.34.1

Initialise empty git Repository in/home/user/devops-git=lab/.git/

on branch Main

No commits Yet

Changes to be committed:

new file : devops.txt

[main (root - commit) 123abc] Initial Devops commit | file changed | insertion (+)

commit 123abc456xyz

Author : user

Date : Sun Feb 23 2026

Initial Devops commit

Program:

```
git --version
```

```
mkdir devops_git_lab
```

```
cd devops_git_lab
```

```
git init
```

```
nano devops.txt
```

```
git status
```

```
git add devops.txt
```

```
git commit -m "Initial Devops commit"
```

```
git log
```

Result:

Thus basic Git version control operations user successfully performance demonstrating how source is tracked and managed in devops.

Ex.No: 04
Date:

Git Version control Repository, Branching and Merging

Aim:

To Initialise a Git repository, commit changes create and merge branches resolve conflict and practices Git push and pull operation using are not repository.

Software Requirements:

1. Ubuntu Linux / window
2. Git
3. GitHub / Gitlab

Algorithm

1. Start the process.
2. Initialise a Git repository.
3. Create and commit files
4. Create and Merge branches.
5. Resolve merge conflict.
6. Push and Pull changes from remote repository.
7. Stop the process show the result.

Output:

Initialised empty Git Repository

[main (root-commit)] Initial commit | file changed | insertion (+)

Switched to branch 'dev'

Switched to branch 'main'

Already up to date

Branch 'main' set up to track remote

Branch 'main'

Everything up to date

Program:

```
git init
```

```
echo "Devops Lab" > file.txt
```

```
git add file.txt
```

```
git commit -m "Initial commit"
```

```
git branch dev
```

```
git checkout dev
```

```
git checkout main
```

```
git merge dev
```

```
git remote add origin <repo-url>
```

```
git push -u origin main
```

```
git pull origin main
```

Result:

Thus, basic git version control operations uses successfully performed demonstrating how source code is tracked and merged in devops.

Ex.No: 05
Date:

Fork, Clone and pull Request using Github

Aim:

To fork a repository clone it locally make change create a pull request and merge changes in a collaboration team environment.

Software Requirements:

1. Git account
2. Git
3. Web browser

Algorithm:

1. Start the process.
2. Fork a repository.
3. Clone repository locally.
4. Make code changes.
5. Push changes to forked repo.
6. Create pull request.
7. Preview and merge changes.
8. Stop the process show the result.

Output:

Cloning into 'repository'

remote : Enumerating objects....

Receiving object : 100%

[main abc 1234] updated file | file changes | insertion (+)

To

<https://github.com//ashok13082804/repository.git>

123abc....abc1234 main -> main

Program:

```
git clone <forked-repo-url>
```

```
cd repository
```

```
nano readme.md
```

```
git add.
```

```
git commit -m "Updated file"
```

```
git push origin main
```

Result:

Thus repository forking cloning pul request creation and merging successfully demonstrated and needed very good perfection result.

Ex.No: 06
Date:

Centralised version control using SVN

Aim:

To understand and perform basic version control operating using SVN (subversion) such as creating a repository checking out files committing, changes and updating the working copy.

Software Requirements:

1. Ubuntu Linux / windows
2. Apache subversion prompt
3. Terminal / command prompt

Algorithm:

1. Start the process.
2. Install SVN.
3. Create an SVN repository.
4. Checkout repository to local system.
5. Create or modify files.
6. Commit changes to SVN Repository.
7. Update working copy.
8. Stop the process show the result.

Output:

Checked out revision 0.

? devops.txt

A devops.txt

Adding devops.txt

Transmitting file data

Committed revision

updating ‘.’

At revision .

Program:

```
sudo apt update
```

```
sudo apt install subversion -y
```

```
svnadmin create /home.user/svnrepo
```

```
svn checkout file:///home/user/svnrepo svn_project
```

```
cd svn_project
```

```
nano devops.txt
```

```
svn status
```

```
svn add devops.txt
```

```
svn commit -m "Initial SVN commit"
```

```
svn update
```

Result:

Thus basic version control operations were successfully performed using SVN demonstrating centralised version control system concept.

Ex.No: 07
Date:

Application Deployment using Docker Compose

Aim:

To define and run a multi container applications using Docker compose with frontend backend and database service.

Software Requirements:

1. Docker
2. Docker compose
3. Ubuntu OS

Algorithm:

1. Start the process.
2. Create Docker compose.yml.
3. Define frontend backend and database.
4. Configure network and dependencies.
5. Run application stack.
6. Verify service.
7. Stop the process show the result.

Output:**Welcome to nginx**

If you see the page this nginx web server is successfully installed and working Further configuration is required.

For online documentation and support please refer to nginx.org

Communicated support is available at nginx.com

Thank You using nginx.

Program:

```
docker-compose.yml
```

```
version: '3'
```

```
services:
```

```
web:
```

```
  image: nginx
```

```
ports:
```

```
  - "8080:80"
```

```
backend:
```

```
  image: node
```

```
db:
```

```
  image: mysql
```

Run Compose

```
docker-compose up -d
```

Result:

Thus the above program is executed successfully and the output is verified.

Ex.No: 08
Date:

Configure Management using Ansible

Aim:

To install and configure nginx on multiple users using ansible playbook demonstrating idempotent.

Software Requirements:

1. Ansible
2. Target Nodes
3. SSH access

Algorithm:

1. Start the process.
2. Configure inventory file.
3. Write Ansible playbook.
4. Execute playbook.
5. Verify night installation.
6. Stop the process show the result.

Output:

Play [web servers]

Task [Gathering Facts]

Ok [local host]

task [Install nginx]

Ok [localhost]

Play Recap

localhost : ok=2 changed=0

unreachable=0 failed=0

Program:

Playbook

- hosts: webservers

become: yes

tasks:

- name: Install nginx

apt:

name: nginx

state: present

Run Playbook

ansible-playbook nginx.yml

Result:

Thus the above program is executed successfully and the output verified.

Ex.No: 09
Date:

Implementing Docker Volume and Docker Networking

Aim:

To create and use docker volumes for data persistence and configure docker network to enable communication between container in micro services.

Software Requirements:

1. Ubuntu Linux / windows
2. Docker
3. Terminal / command prompt
4. Internet connection

Algorithm:

1. Start the process.
2. Install and verify the docker.
3. Create and run the docker volume.
4. Create docker network and run the multiple container same network.
5. Verify container communication.
6. Stop the process show the result.

Output:

```
Ping webs (172.19.0.3) 56 (84) bytes of data
64 byte from web3. devops_network (172.19.03): 1cmp-
seq = 1 H1 = 64      time = 1.02ms
64 byte from web3. devops_network (172.19.03): 1cmp-
seq = 2 H1 = 64      time = 0.17ms
64 byte from web3. devops_network (172.19.03): 1cmp-
seq = 3 H1 = 64      time = 0.09ms
64 byte from web3. devops_network (172.19.03): 1cmp-
seq = 4 H1 = 64      time = 0.11ms
- - - web 3 ping statistics
32 packets transmitted, 22 received 0% packet loss
rat min/avg/max/mdw = 0.067/0.68/2.547/0.52ms
```

Program:

```
docker --version
```

```
docker volumes create devops_volume
```

```
docker run -dit --name web1 -v devops_volume:/usr/share/nginx/html  
nginx
```

```
docker exec -it web1 bash
```

```
echo "Docker Volume Test" > /usr/share/nginx/html/index.html  
exit
```

```
docker network create devops_network
```

```
docker run -dit --name web2 --network devops_network nginx
```

```
docker run -dit --name web3 --network devops_network nginx
```

```
docker exec -it web2 ping web3
```

```
docker stop web1
```

```
docker rm web1
```

```
docker run -dit --name web1_new -v
```

```
devops_volume:/usr/share/nginx/html nginx
```

Result:

Thus the above program is executed successfully and the output is verified.

Ex.No: 10
Date:

Configuration Management using Ansible

Aim:

To install and configure nginx on multiple server using ansible playbook demonstrating idempotency.

Software Requirements:

1. Ansible
2. Target Nodes
3. SSH access

Algorithm:

1. Start the process.
2. Configure inventory file.
3. Write Ansible playbook.
4. Execute playbook.
5. Verify nginx installation.
6. Stop the process show the result.

Output:

PLAY [webserver] * * * * *

TASK [Gathering Facts] * * * * *

TASK [Install nginx] * * * * *

PLAY RECAP* * * * *

localhost : ok = 2 changed = 0 unreachable = 0 failed = 0

skipped = 0 reused = 0 ignored = 0

Program:

Playbook

- hosts: webservers

become: yes

tasks:

- name: Install nginx

apt:

name: nginx

state: present

Run playbook

ansible-playbook nginx.yml

Result:

Thus the above program is executed successfully and the output is verified.

Ex.No: 11
Date:

Kubernetes Deployment using Minikube

Aim:

To deploy a multi service application in Minikube expose it using Kuberctl services and scale deployment.

Software Requirements:

- 1.Kubernete
2. Docker
3. Minikube

Algorithm:

1. Start the process.
2. Start Minikube.
3. Deploy application.
4. Expose service.
5. Scale deployment.
6. Stop the process show the result.

Output:

ashok@ashok : ~\$ kubectl get services

Name	type	Cluster-IP	External	Ports	Age
app	Nodeport	10.95.57.175	<none>	80:31591/TCP	65ms
Kubernetete	ClusterIP	10.96.0.1	<none>	443/TCP	82ms

ashok@ashok : ~\$ get pods

Name	Ready	Status	Restarts	Age
app-6b97cd8cbd-52826	1/1	Running	0	7m29ms
app-6b97cd8cbc-j4ft4	1/1	Running	0	6m57ms
app-6b97cd8cbd-sqkz4	1/1	Running	0	6m57ms

ashok@ashok : ~\$ kubernete get deployment

Name	Ready	Up to date	Available	Age
app	3/3	3	3	7m47ms

Program:

start Minikube

minikube start

Deploy App

kubectl create deployment app - -image=nginx

kubectl expose deployment app - -type=NodePort - -port=80

Scale

kubectl scale deployment app - -replica=3

Result:

Thus the above program is executed successfully and the output is verified.

Ex.No: 12
Date:

Kubernetes Deployment Rolling update and Rollback

Aim:

To deploy an application in kubectl using Deployment perform rolling updates and rollback to a previous version when required.

Software Requirements:

- 1.Kubernetete
2. Docker
3. Minikube
4. Ubuntu Linux / Window

Algorithm:

1. Start the process.
2. Start Minikube cluster.
3. Create a Deployment using service and without service.
4. Perform rolling update.
- 5.Rollback to previous version.
6. Stop the process show the result.

Output:

```
ashok@ashok : ~$ kubectl get service
```

Name	Ready	Status	Restart	Age
app-6b97cd8cbd-52826	1/1	Running	1 (90s ago)	17m
app-6b97cd8cbc-j4ft4	1/1	Running	1 (90s ago)	16m
app-6b97cd8cbd-sqkz4	1/1	Running	1 (90s ago)	15m
webapp-65f665df676-mkbbk	1/1	Running	0	67s

```
ashok@ashok : ~$ kubectl rollout status deployment / web app
```

```
deployment "web app" successfully rolled out
```

Program:

```
#!/bin/bash
minikube start
kubectl create deployment web app - -image = nginx:1.21
kubectl expose deployment web app - - type = NodePort - - port = 80
kubectl get pods
kubectl set image deployment/webapp nginx-nginx:1.23
kubectl rollout status deployment / webapp
kubectl rollout undo deployment / webapp
kuberctl describe deployment web app
```

Result:

Thus the above program is executed successfully and the output verified.

Ex.No: 13
Date:

CI / CD Pipeline using Jenkins

Aim:

To create a jenkins pipeline to built, test and push docker images automatically upon Git push.

Software Requirements:

1. Jenkins
2. Docker
3. Github

Algorithm:

1. Start the process.
2. Create Jenkins pipeline.
3. Clone source code.
4. Run unit test.
5. Build docker image.
6. Push image to docker hub.
7. Stop the process show the result.

Output:

jenkins service - jenkins continuous Integration Server

Loaded : loaded (/ lib /system a/ system /jenkins.services,enable)

Active : active (running) since Tue 2026-03-31 10:15:4LIST

Main PID : 12345 (java)

Task : 42 (limit : 4915)

Memory : 2563M

cpu : 12.3453

croup : /system.slice/jenkins.service

12345 /usr/bin/java-jar/usr/shai/java/jenkins.war

Program:

Jenkinsfile

```
pipeline {  
  stages {  
    stage('Build') {  
      steps {  
        sh 'docker build -t my app .'      }  
    }  
  }  
}
```

Output:

Thus the above program is executed successfully and the output is verified.

Ex.No: 14
Date:

Monitoring using Prometheus and Grafana

Aim:

To monitor a microservice using prometheus visualise using grafana.

Software Requirements:

1. Prometheus
2. Docker
3. Grafana

Algorithm:

1. Start the process.
2. Deploy Microservice.
3. Configure prometheus.
4. Scrape metris.
5. Configure Grafana dashboard.
6. Visualise metris.
7. Stop the process show the result.

Output:

Unable to final image 'prom / prometheus : 'atut' locally

l atut : pulling from prom / prometheus

a1 b2 c3 d4 e5 f6 : pull completed

a1 b2 c3 d4 e5 f6 : pull completed

Digest : sha256 : abcd1234efgh5678ijkl9012mnop3456qrst8901

Status : Download new image for prom / prometheus 1 atut

da+8c 7b6a5e4d3c2b1a0987654321fedcba1234567890abcd

Unable to final image 'graf / grafana : 'atut' locally

l atut : pulling from graf / grafana

c1 d2 e3 f4 g5 h6 : pull completed

d2 e3 f4 g5 h6 i7 : pull completed

Digest : sha256 : abcd1234efgh5678ijkl9012mnop3456qrst8901

Status : Download new image for graf / grafana 1 atut

da+8c 7b6a5e4d3c2b1a0987654321fedcba1234567890abcd

Program:

Run Prometheus

```
docker run -d -p 9090 : 9090 prom / prometheus
```

Run Grafana

```
docker run -d -p 3000 : 3000 grafana / grafana
```

Result:

Thus the above program is executed successfully and the output is verified.

Ex.No: 15
Date:

Centralised Logging for MicroServices using ELK stack

Aim:

To count, process and visualise logs from a micro services using the ELK stack.

Software Requirements:

1. Ubuntu Linux / Windows
2. Docker and Docker Compose
3. Basic micro services application
4. Internet Connection

Algorithm:

1. Start the process.
2. Instance and verify Docker and Docker Compose.
3. Deploy elastic search, Logstash, and kibana container.
4. Configure Microservices logging.
5. View logs in kibana dashboard.
6. Stop the process show the result.

Output:

Docker version 26.1.0 built 1234567

Docker compose version v2.24.5

Creation network "default default" with the default

pulling Elasticsearch (elasticsearch 7.17.0)...

7.17.0 : pulling from library / elasticsearch

Digest :sha256 : abcd1234efgh5678ijkl9012mnop3456qrst7890uvwxyz1234yz56

Status : Downloaded never image for elastic : 7.17.0

Access kibana at: **https://localhost**:5601

Program:

```
#!/bin/bash

docker --version

docker-compose --version

cat <<EOF > docker-compose.yml
version: '3'
services:
  elasticsearch:
    image: elasticsearch : 7.17.0
    environment:
      - discovery.type = single-node
    ports:
      - "9200 : 9200"
  kibana:
    image : kibana : 7.17.0
    ports :
      - "5601 : 5601"
    depends_on:
      - elasticsearch
EOF

docker-compose up -d

docker run --rm busybox echo "Devops Log Test"

echo "Access Kibana at : http://localhost:5601"
```

Result:

Thus the above program is executed successfully and the output is verified.

